

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Jesssenia Belen Cruz Rojas

PARALELO: "A"

SEMESTRE: QUINTO

ACTIVIDAD: Activida#1: Estudio de Caso – Sistemas de Gestion de Incidentes (Help Desk)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 1

Actividad#1: Estudio de Caso – Sistemas de Gestión de Incidentes (Help Desk)

1. INTRODUCCIÓN

Actualmente, los centros de datos requieren sistemas eficientes que permitan gestionar incidentes técnicos relacionados con hardware, software y redes. Un sistema Help Desk facilita el registro, control y seguimiento de fallos técnicos mediante tickets organizados.

El presente trabajo tiene como finalidad diseñar la arquitectura lógica de un sistema de gestión de incidentes aplicando patrones de diseño de software, principios SOLID y el modelo Gitflow para el control profesional de versiones en GitHub.

2. OBJETIVO GENERAL

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles...

3. ANÁLISIS DEL PROBLEMA

En un Data Center existen diferentes tipos de incidentes técnicos que afectan la disponibilidad de los servicios tecnológicos. Actualmente, la gestión manual de incidencias provoca retrasos en la atención, desorganización y dificultades en el seguimiento de problemas.



Por ello, se propone un sistema Help Desk que permita:

- Reportar incidentes.
- Clasificar fallos.
- Asignar técnicos especializados.
- Gestionar estados de resolución.
- Mantener control de versiones mediante Gitflow.

4. ACTORES PRINCIPALES

Actor	Función
Usuario Final	Reporta incidentes y consulta estados
Técnico de Soporte	Atiende y resuelve incidentes
Administrador	Gestiona usuarios y asignaciones

5. REQUERIMIENTOS DEL SISTEMA

5.1 Requerimientos Funcionales

Crear tickets.

Categorizar incidentes.

Asignar técnicos.

Actualizar estados.

Generar notificaciones.



Consultar historial.

5.2 Requerimientos No Funcionales

Escalabilidad.

Mantenibilidad.

Seguridad.

Trazabilidad de versiones.

Disponibilidad.

6. DISEÑO DEL SISTEMA

El sistema fue diseñado utilizando arquitectura MVC (Model – View – Controller), permitiendo separar la lógica del negocio, la interfaz gráfica y el control de procesos.

Además, se implementó una arquitectura modular utilizando patrones de diseño para mejorar la reutilización y mantenimiento del sistema.

7. IMPLEMENTACIÓN DE GITFLOW

Para el desarrollo del sistema Help Desk se utilizó GitHub bajo el modelo Gitflow, permitiendo un control organizado de versiones y un flujo de trabajo profesional.

7.1 Estructura de ramas

Rama	Función
master	Versión estable del sistema



develop	Desarrollo principal
feature/diagramas	Desarrollo de diagramas UML

7.2 Comandos utilizados

```
git init
```

```
git checkout -b develop
```

```
git checkout -b feature/diagramas
```

```
git add .
```

```
git commit -m "Actualizacion diagramas UML"
```

```
git push origin master
```

7.3 Enlace del repositorio GitHub

<https://github.com/jcruz0393-pixel/HelpDeskDS.git>

7.4 Evidencias Gitflow

- **Repositorio principal GitHub**



Universidad

Técnica de Manabí

GitHub profile page for **jacruz0393-pixel**. The profile shows two repositories:

- HelpDeskDS** (Public): Actividad 1 - Sistema Help Desk - Desarrollo de Sistemas Informáticos. Updated 17 minutes ago.
- helpdesk_actividad_DS** (Public): Actividad 1 - Sistema Help Desk - Desarrollo de Sistemas Informáticos. Updated 2 days ago.

User information: jacruz0393-pixel, Email: jacruz0393@utm.edu.ec, Joined 2 days ago.

- Ramas Gitflow

GitHub repository page for **HelpDeskDS** (Public). The repository is managed using Gitflow. The current branch is **main**. The repository structure shows 6 branches and 0 tags.

Repository description: Actividad 1 - Sistema Help Desk - Desarrollo de Sistemas Informáticos.

Repository statistics: 0 stars, 0 watching, 0 forks.



- Historial de commits

```
552 diagrams/Diagrama UML.drawio
@@ -0,0 +1,552 @@
1  + cmxfile host="Electron" agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
  like Gecko) draw.io/28.2.8 Chrome/149.0.7339.240 Electron/38.4.0 Safari/537.36" version="28.2.8"
  pages="5">
2  + <diagram name="DIAGRAMA DE CASOS DE USO" id="BM2-x0cc0gQESPvc63q">
3  + <mxGraphModel dx="1489" dy="894" grid="1" gridSize="10" guides="1" tooltips="1" connect="1"
  arrows="1" fold="1" page="1" pageScale="1" pageWidth="827" pageHeight="1169" math="0" shadow="0">
4  + <root>
5  + <mxCell id="0" />
6  + <mxCell id="1" parent="0" />
7  + <mxCell id="Urw-p0gKs3PQqMKv1_3" value="<h1 class="h1 mb-0"
  id="yul_1_18_1_1_1778683794871_96" style="box-sizing: border-box; margin-top: 0px;
  line-height: 1.2; color: rgb(29, 33, 37); font-family: -apple-system, BlinkMacSystemFont,
  &quot;Segoe UI&quot;, Roboto, &quot;Helvetica Neue&quot;, Arial, &quot;Noto
  Sans&quot;, &quot;Liberation Sans&quot;, sans-serif, &quot;Apple Color
  Emoji&quot;, &quot;Segoe UI Emoji&quot;, &quot;Segoe UI Symbol&quot;,
  &quot;Noto Color Emoji&quot;; font-variant-ligatures: normal; font-variant-caps: normal;
  letter-spacing: normal; orphans: 2; text-indent: 0px; text-transform: none; widows: 2; word-spacing:
  0px; &quot;webkit-text-stroke-width: 0px; white-space: normal; background-color: rgb(255, 255, 255); text-
  decoration-thickness: initial; text-decoration-style: initial; text-decoration-color: initial;
  margin-bottom: 0px !important;&quot;&quot;&quot; font style=&quot;font-size: 26px;&quot;&quot;&quot; DIAGRAMA DE
  CASOS DE USO&lt;/font&quot;&quot;&quot; font style=&quot;font-size: 26px;&quot;&quot;&quot;
  style="text;whiteSpace-wrap;html-1;fillColor=#d80073;fontColor=#ffffff;strokeColor=#a50040;fontStyle=
  3;align:center;\" parent='1' vertex='1'>
8  + <mxGeometry x="120" y="20" width="580" height="40" as="geometry" />
9  + </mxCell>
10 + <mxCell id="Urw-p0gKs3PQqMKv1_4" value="USUARIO"
  style="shape=umlActor;verticalLabelPosition=bottom;verticalAlign=top;html-1;fillColor=#b1ddff;strokeC
  olor=#18739e;" parent="1" vertex="1">
11 + <mxGeometry x="100" y="120" width="60" height="60" as="geometry" />
12 + </mxCell>
```

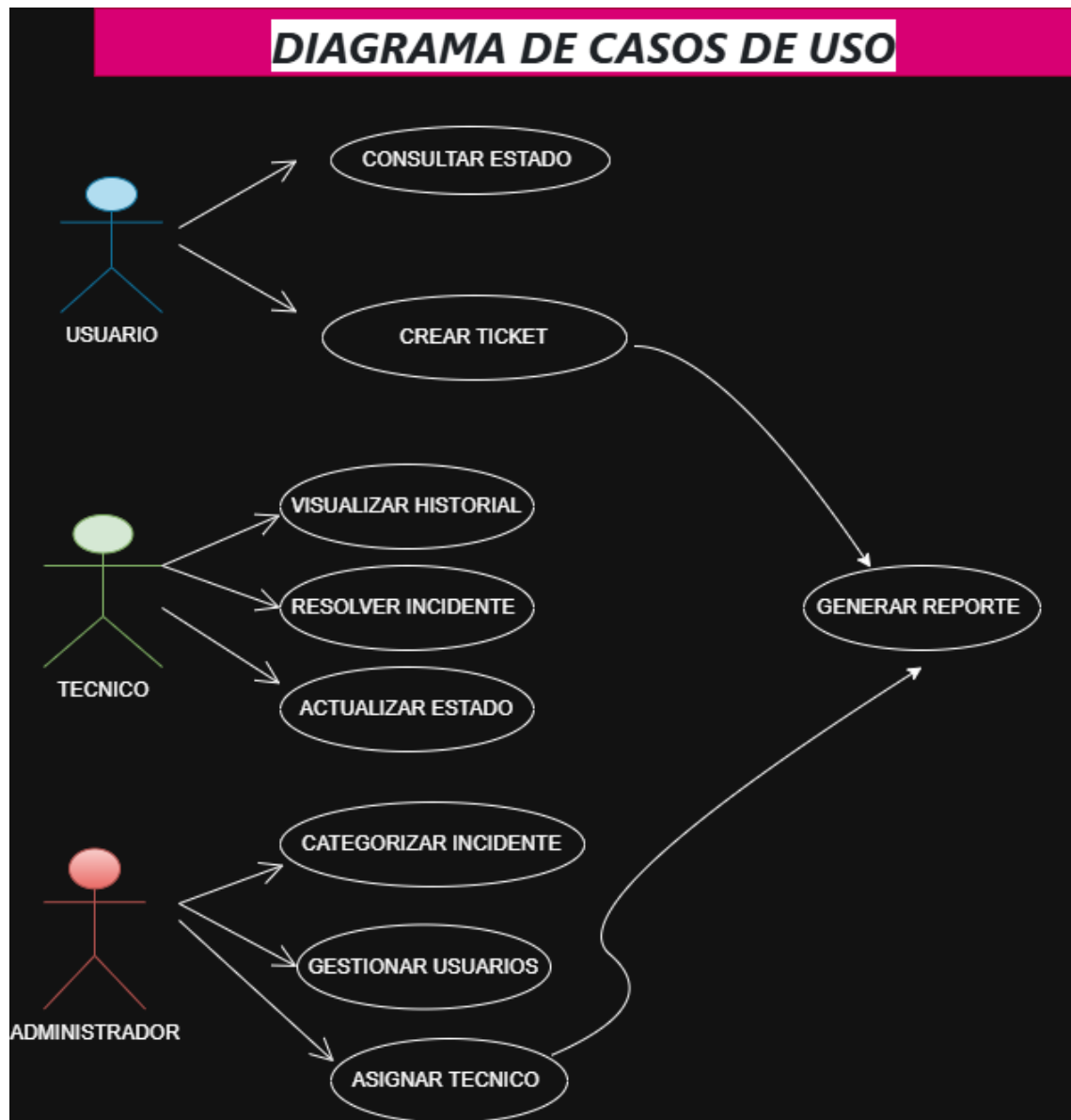
8. DIAGRAMAS UML

8.1 Diagrama de Casos de Uso

Este diagrama representa las interacciones entre los actores principales del sistema Help Desk.

- Actores involucrados
- Usuario Final
- Técnico de Soporte
- Administrador
- Funciones principales
- Crear tickets
- Consultar estado
- Asignar técnicos

- Resolver incidentes



8.2 Diagrama de Clases UML

El diagrama de clases representa la estructura lógica del sistema y la relación entre las clases principales.

Además, evidencia la implementación de patrones de diseño y principios SOLID.

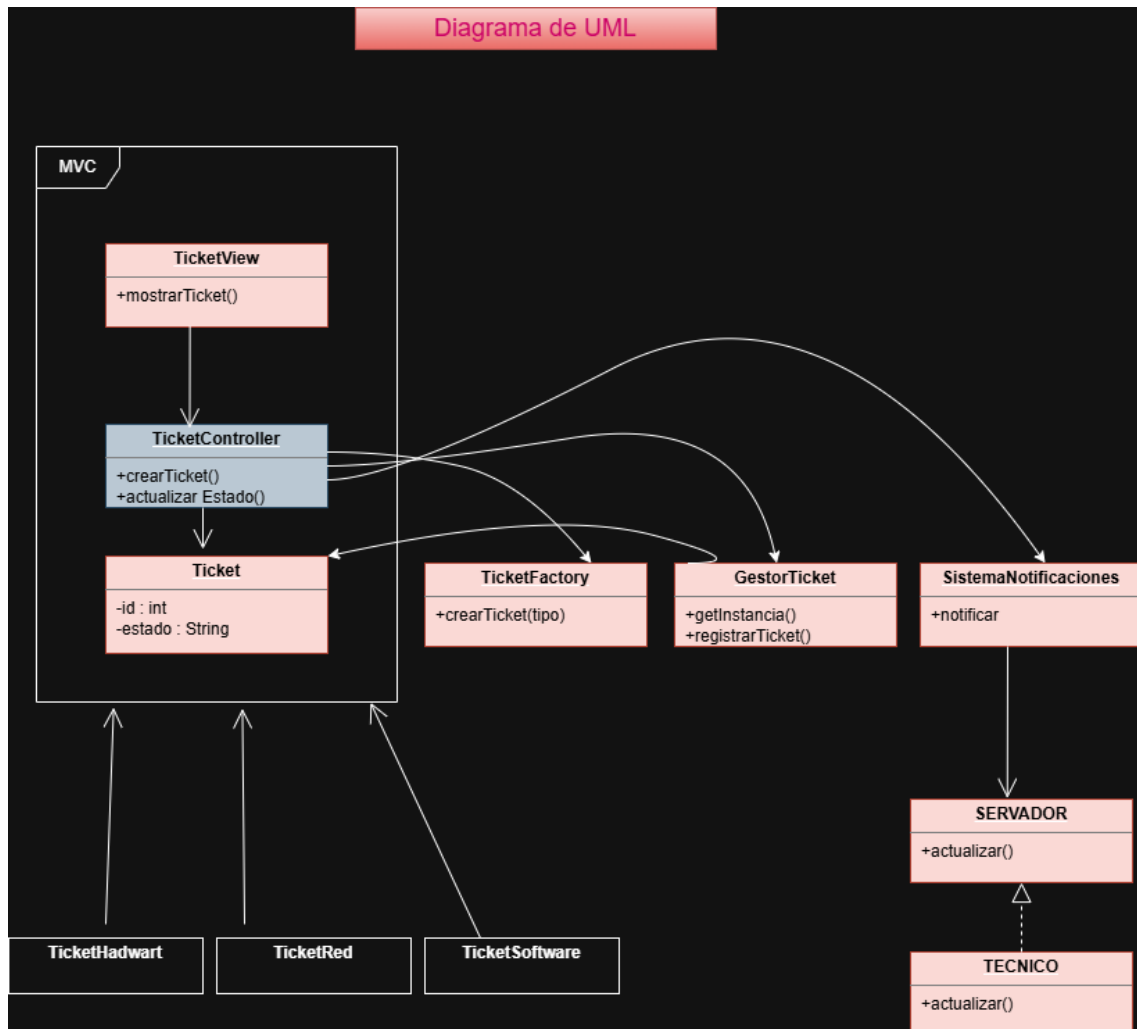


Patrones implementados

Patrón	Clase
Singleton	GestorTickets
Observer	SistemaNotificaciones
Factory Method	TicketFactory
MVC	View, Controller y Model



Diagrama de Clases UML



8.3 Diagrama Secuencial – Usuario

Este diagrama muestra el proceso realizado por el usuario al momento de reportar un incidente.

Flujo principal

El usuario crea un ticket.

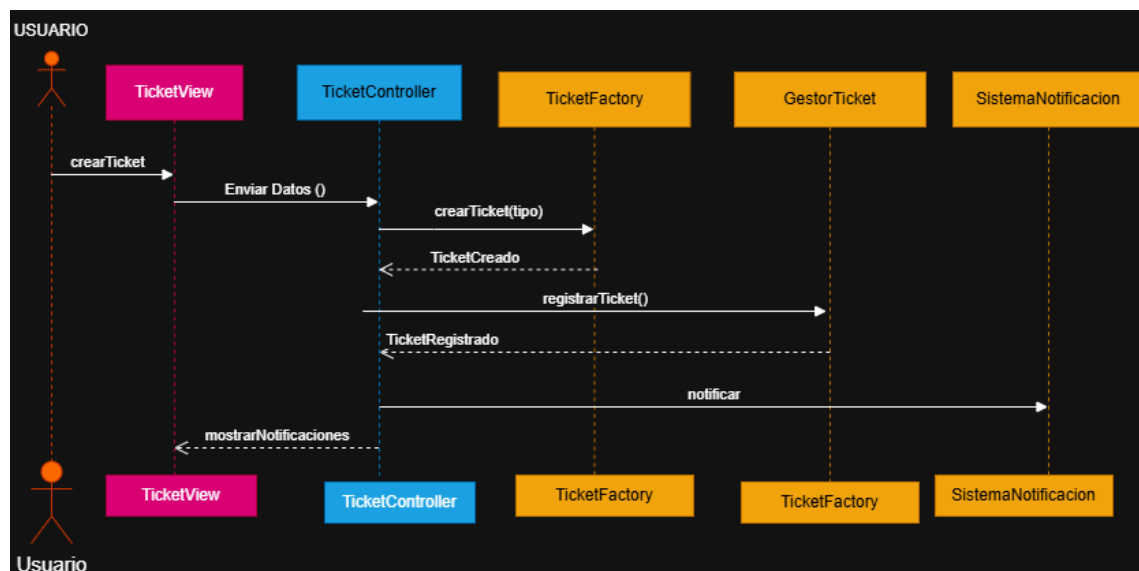


El controlador procesa la solicitud.

Se genera el ticket.

1. El sistema registra la incidencia.
2. Se notifica al técnico.

Diagrama Secuencial Usuario



8.4 Diagrama Secuencial – Técnico

Este diagrama representa el proceso realizado por el técnico de soporte para atender incidentes.

Flujo principal

El técnico recibe una notificación.

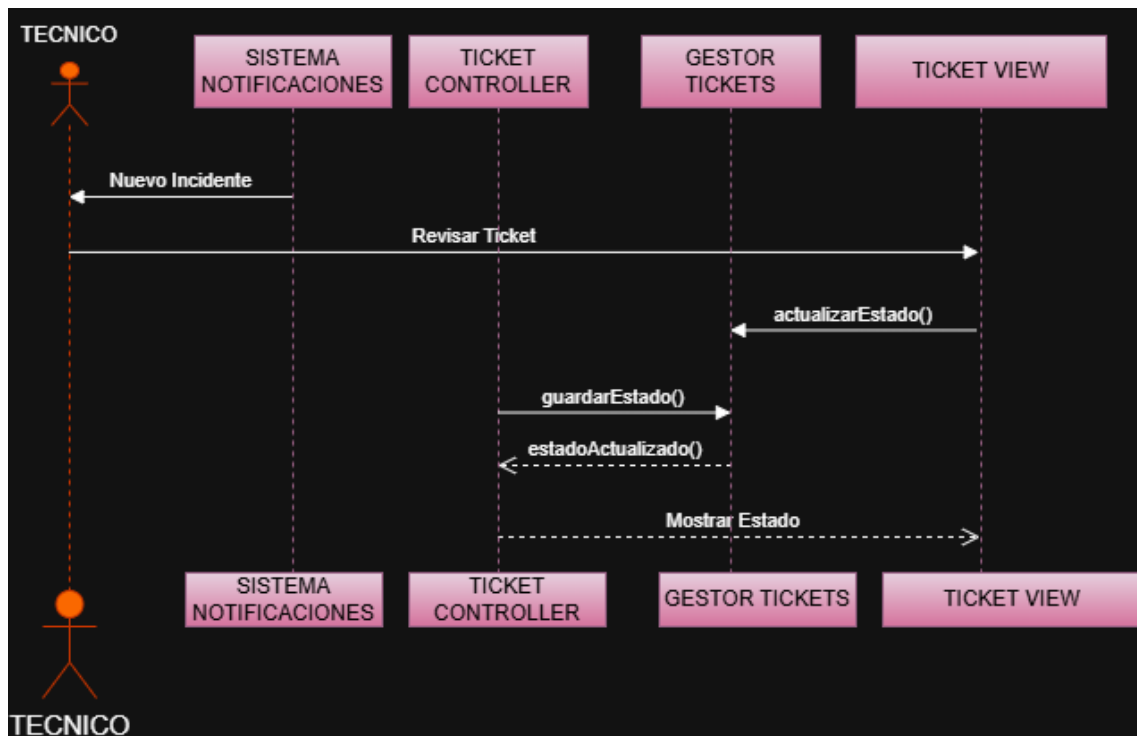
Revisa el ticket.



Actualiza el estado.

El sistema guarda los cambios.

- **Diagrama Secuencial Técnico**



8.5 Diagrama de Arquitectura

El diagrama de arquitectura representa la estructura general del sistema Help Desk utilizando MVC y patrones de diseño.

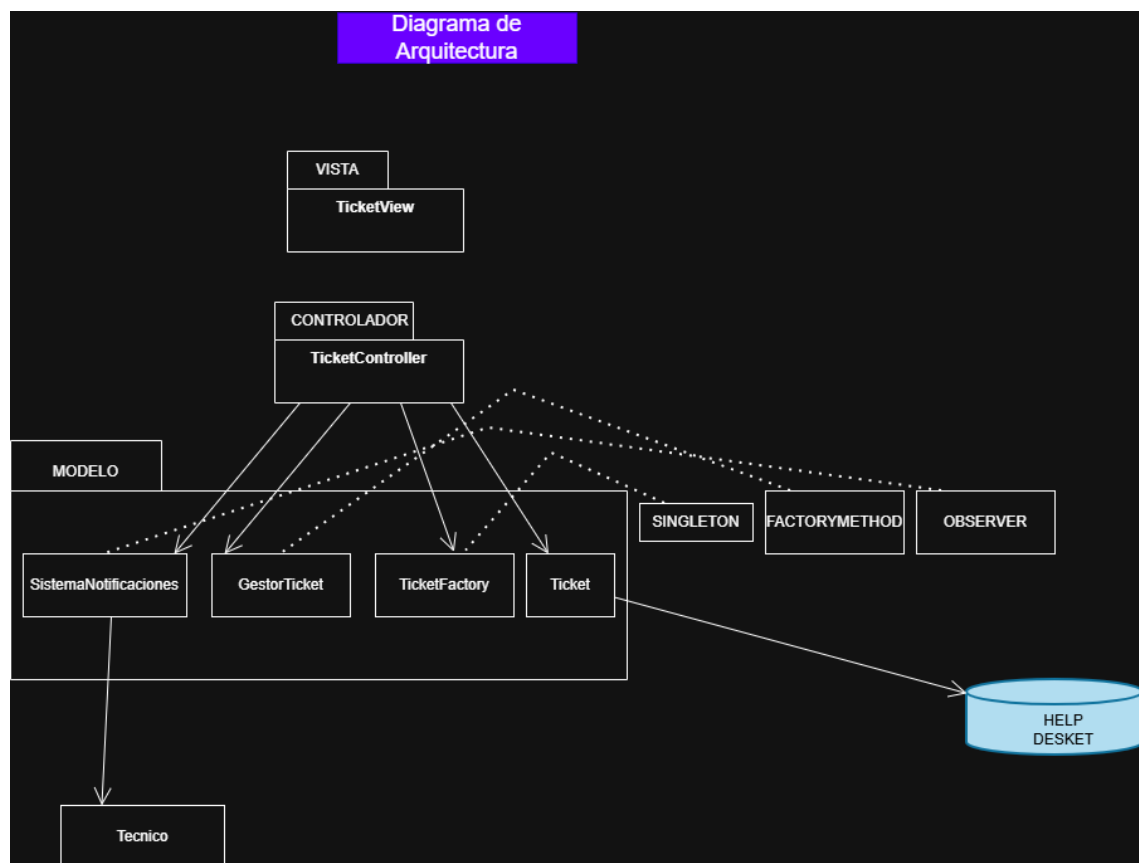
Componentes principales

Componente	Función
Vista	Interfaz de usuario



Controlador	Gestiona procesos
Modelo	Administra datos
Base de Datos	Almacena información

- Diagrama de Arquitectura



9. APLICACIÓN DE PATRONES DE DISEÑO

Los patrones de diseño fueron implementados para garantizar organización, reutilización de código y mantenibilidad dentro del sistema Help Desk.

9.1 Singleton

Propósito



Garantizar una única instancia del gestor principal del sistema.

Aplicación

El patrón Singleton se utilizó en la clase GestorTickets para centralizar el manejo de tickets y conexión de datos.

Beneficio

Evita múltiples instancias innecesarias y mejora el control centralizado.

9.2 Observer

Propósito

Permitir notificaciones automáticas a técnicos.

Aplicación

El sistema notifica automáticamente a los técnicos cuando se genera un nuevo incidente.

Beneficio

Mejora la comunicación entre módulos y automatiza alertas.

9.3 Factory Method

Propósito

Crear diferentes tipos de tickets.

Aplicación



Se implementó para generar tickets:

- Hardware
- Red
- Software

Beneficio

Facilita agregar nuevos tipos de incidentes sin modificar la lógica principal.

9.4 MVC (Model – View – Controller)

Propósito

Separar interfaz, lógica y datos.

Aplicación

Componente	Función
Model	Gestión de datos
View	Interfaz gráfica
Controller	Control del sistema

Beneficio

Mejora organización, escalabilidad y mantenimiento.

10. ARQUITECTURA GENERAL DEL SISTEMA

El sistema se encuentra dividido en las siguientes capas:



Vista

Interfaz para usuarios y técnicos.

Controlador

Gestiona solicitudes y procesos.

Modelo

Administra tickets, usuarios y notificaciones.

Base de Datos

Almacena información de incidentes y usuarios.

11. PRINCIPIOS SOLID APLICADOS

Single Responsibility Principle

Cada clase tiene una sola responsabilidad.

Open/Closed Principle

El sistema permite agregar nuevos tipos de tickets sin modificar clases existentes.

Liskov Substitution Principle

Las clases hijas pueden reemplazar clases base sin afectar el sistema.

Interface Segregation Principle

Las interfaces fueron diseñadas para funciones específicas.



Dependency Inversion Principle

Las clases dependen de abstracciones y no de implementaciones concretas.

12. JUSTIFICACIÓN TÉCNICA

La utilización de patrones de diseño permitió mejorar la organización y reutilización del sistema.

El uso de Gitflow facilitó el control de versiones mediante ramas independientes para desarrollo y producción.

La arquitectura MVC mejoró la separación de responsabilidades y permitió una estructura escalable.

13. CONCLUSIONES

- El sistema Help Desk permite gestionar incidentes técnicos de manera organizada.
- La implementación de Singleton, Observer y Factory Method mejora la mantenibilidad y reutilización.
- Gitflow proporciona un flujo de trabajo profesional y ordenado.
- La arquitectura MVC mejora la separación de responsabilidades.
- Los principios SOLID fortalecen la escalabilidad del sistema.

14. RECOMENDACIONES

- Mantener documentación actualizada.
- Utilizar commits frecuentes y descriptivos.



- Validar diagramas antes de subirlos al repositorio.
- Aplicar buenas prácticas de programación.
- Mantener organización en ramas Gitflow.

15. BIBLIOGRAFÍA

<https://online.utm.edu.ec/>

Draw.io Documentation. (2026). *Diagramming Software and Flowchart Maker*.

Recuperado de:

<https://app.diagrams.net/>

Repositorio GitHub del proyecto Help Desk DS. Recuperado de:

<https://github.com/jcruz0393-pixel/HelpDeskDS>